

Modeling Dynamical Systems in the Situation Calculus

Raymond Reiter
Department of Computer Science
University of Toronto
and
The Canadian Institute for Advanced Research

<http://www.cs.toronto.edu/~cogrobo/>

Cognitive Robotics

- Theory and implementation of agents that reason, act and perceive in changing, incompletely known, unpredictable environments. Reasoning about:
 - Goals.
 - Actions.
 - Time, continuous events.
 - When to perceive and what to look for.
 - The cognitive states of other agents.
- In short, cognitive robotics is concerned with integrating reasoning, perception and action within a uniform theoretical and implementation framework.
- Contrast with virtually all current robotics research.
 - Sensory processing.
 - Path planning.
 - Manipulator design and control.
 - Reactive agents, cockroach AI.
- Requires a sophisticated theory of actions, including perceptual actions, complex actions, concurrency, time, exogenous events, etc.

This Talk

- Describe the *situation calculus*, a first order language for representing dynamical systems.
- Indicate how one axiomatizes world dynamics in the sitcalc.
- Describe the *frame problem* and a proposed solution.
- GOLOG, a high level logic programming language for specifying and implementing complex behaviours.
- A theorem-proving interpreter for GOLOG, and a Prolog implementation.
- Open world GOLOG.
- Induction for proving properties of GOLOG programs.

The Situation Calculus (McCarthy, 1963)

- A first order language for representing dynamically changing worlds; all changes are the result of named *actions*.
- A *situation*: a first order term denoting a possible world history (a sequence of actions).
- These sequences are represented using a binary function symbol *do*: $do(\alpha, s)$ denotes the history resulting from adding the new action α to the history s .

- Actions are first order terms:

$put(x, y)$ = put object x on object y .

$do(put(A, B), s)$ = that situation resulting from placing A on B when the current situation is s .

$do(putdown(A), do(walk(L), do(pickup(A), S_0)))$

denotes the sequence of actions

$[pickup(A), walk(L), putdown(A)]$.

- *Fluent* = relation or function whose values may vary from situation to situation.

– Denoted by predicate/function symbol taking a situation term as one of its arguments.

$closeTo(x, y, s)$: x is close to y in situation s .

$pos(x, s)$: x 's position in situation s .

Axiomatizing Actions in the Sitcalc

- Actions have *preconditions*:

$$Poss(pickup(x), s) \equiv [(\forall z) \neg holding(z, s)] \wedge nextto(x, s) \wedge \neg heavy(x).$$

$$Poss(repair(x), s) \equiv hasglue(s) \wedge broken(x, s).$$

- World dynamics are specified by *effect axioms*.

These specify the effects of a given action on the fluents – the *causal laws* of the world.

$$Poss(drop(x), s) \wedge fragile(x, s) \supset broken(x, do(drop(x), s)).$$

$$Poss(repair(x), s) \supset \neg broken(x, do(repair(x), s)).$$

$$Poss(paint(x, c), s) \supset colour(x, do(paint(x, c), s)) = c.$$

The Frame Problem (McCarthy and Hayes)

- Axioms other than effect axioms are required:

- *Frame axioms.*
- Specify the action *invariants* of the domain: those fluents unaffected by the action.
- Dropping things does not affect an object's colour:

$$\begin{aligned} Poss(drop(x), s) \wedge colour(y, s) = c \\ \supset colour(y, do(drop(x), s)) = c. \end{aligned}$$

- Not breaking things:

$$\begin{aligned} Poss(drop(x), s) \wedge \neg broken(y, s) \wedge [y \neq x \vee \neg fragile(y, s)] \\ \supset \neg broken(y, do(drop(x), s)). \end{aligned}$$

- *Frame Problem:*

- Vast number of frame axioms; only relatively few actions will affect the truth value of a given fluent.
- All other actions leave the fluent invariant.
- An object's colour is not changed by picking things up, opening a door, turning on a light, electing a new prime minister of Canada, etc. etc.

- $\sim 2 \times \mathcal{A} \times \mathcal{F}$ frame axioms.

What Counts as a Solution to the FP?

- Suppose an axiomatizer succeeds in writing down **all** the effect axioms for some application domain:

For each predicate fluent F and each action A which can cause F 's truth value to change, an axiom of the form

$$Poss(A, s) \wedge R(s) \supset (\neg)F(do(A, s)).$$

For each functional fluent f and each action A which can cause f 's value to change, an axiom of the form

$$Poss(A, s) \wedge R(y, s) \supset f(do(A, s)) = y.$$

- Want a systematic procedure for generating, from these effect axioms, all the frame axioms.
- Also want a *parsimonious* representation for the frame axioms (because in their simplest form, there are too many of them).

The FP: A Simple Solution (Sometimes)

R. Reiter. The Frame Problem in the Situation Calculus: A Simple Solution (Sometimes) and a Completeness Result for Goal Regression. *Artificial Intelligence and Mathematical Theory of Computation: Papers in Honor of John McCarthy*, 1991.

- Based on earlier ideas of Pednault, Haas and Schubert.
- Start with the effect (causal) axioms:

$$Poss(drop(x), s) \wedge fragile(x, s) \supset broken(x, do(drop(x), s)).$$

$$Poss(repair(x), s) \supset \neg broken(x, do(repair(x), s)).$$

- Perform a simple, efficient, syntactic transformation to obtain a *successor state axiom* for *broken*:

$$Poss(a, s) \supset [broken(x, do(a, s)) \equiv fragile(x, s) \wedge a = drop(x) \vee broken(x, s) \wedge a \neq repair(x)].$$

- Analogous to the Clark completion of a logic program.
- Yields simple, extremely compact axioms.
- Caveat: The solution
 - applies to *deterministic, primitive* actions only.
 - does not address the *ramification/qualification* problems.

Primitive, Deterministic Actions Only

- As yet, we have no constructs for complex actions; all actions are *primitive*.
- Conditional actions:

if *car_in_driveway* **then** *drive* **else** *walk*.

- Iterative actions:

clear_table $\triangleq$ **while** $[(\exists block) on_table(block)]$ **do** *remove_a_block*.

- Nondeterministic actions:

remove_a_block $\triangleq$
 $(\pi block)[pickup(block); put_on_floor(block)].$

- Recursive actions: If *down* means move an elevator down one floor, define $d(n)$, meaning move an elevator down n floors.

proc $d(n)$
 if $n = 0$ **then** *no_op*
 else *down*; $d(n - 1)$
end

A Theory of Complex Actions

- A way of defining complex actions in terms of “easy” (wrt the frame problem) *primitive actions*.
- Operators for defining complex actions:
 - Sequence (Do action A , followed by action B).
 - Test actions (Is p true in the current situation?)
 - While loops.
 - Conditionals (**if then else**).
 - Nondeterministic choice of actions (Do A or do B).
 - Nondeterministic choice of arguments to actions (Kick something: $(\pi x)kick(x)$).
 - Procedures: (Giving names to complex actions; recursion).
- This results in a novel programming language for dynamical systems and high level robot control.

SC Definitions of Complex Action Constructors

$Do(a, s, s')$ – Doing action a in situation s leads to a situation s' .

- Primitive actions a :

$$Do(a, s, s') \stackrel{def}{=} Poss(a, s) \wedge s' = do(a, s).$$

- Sequence:

$$Do([a_1; a_2], s, s') \stackrel{def}{=} (\exists s''). Do(a_1, s, s'') \wedge Do(a_2, s'', s').$$

- Nondeterministic choice of two actions:

$$Do([a_1 \mid a_2], s, s') \stackrel{def}{=} Do(a_1, s, s') \vee Do(a_2, s, s').$$

- Nondeterministic choice of action parameters:

$$Do((\pi x)a(x), s, s') \stackrel{def}{=} (\exists x) Do(a(x), s, s').$$

- Conditional actions:

$$\mathbf{if } p \mathbf{ then } a_1 \mathbf{ else } a_2 \stackrel{def}{=} [p?; a_1] \mid [\neg p?; a_2].$$

- While loops:

$$\mathbf{while } p \mathbf{ a} \stackrel{def}{=} [p?; a]^*; \neg p?.$$

- Tests:

$$Do(p?, s, s') \stackrel{def}{=} p[s] \wedge s = s'.$$

Semantics of Complex Actions (Continued)

- Nondeterministic iteration: Execute a 0 or more times.

$$\begin{aligned} Do(a^*, s, s') &\stackrel{def}{=} \\ &(\forall P).[(\forall s_1)P(s_1, s_1)] \wedge \\ &\quad [(\forall s_1, s_2, s_3).P(s_1, s_2) \wedge Do(a, s_2, s_3) \supset P(s_1, s_3)] \\ &\quad \supset P(s, s'). \end{aligned}$$

Doing action a 0 or more times takes you from s to s' iff (s, s') is in the smallest set such that (s_1, s_1) is in the set for all situations s_1 , and whenever (s_1, s_2) is in the set, and doing a in situation s_2 takes you to situation s_3 , then (s_1, s_3) is in the set.

Transitive closure requires second order logic.

- Procedures, including recursion: a second order situation calculus version of the usual Scott-Strachey *least fixed point* semantics for standard programming languages.

Remarks

- These definitions are macros; complex actions are completely eliminated by macro expansion.
- Program execution is described by a (second order) formula of the sitcalc.
- Purely logical, situation calculus semantics. Extremely compact specification.
- With the exception of nondeterministic iteration and procedures, this sitcalc semantics borrows from *dynamic logic*.
- Intuitively, $Do(program, s, s')$ means that s' is one of the situations reached by performing *program*, starting in s .
- $Do(program, s, s')$ is an abbreviation (macro) for a formula of the situation calculus that says exactly this.
- So, to execute *program*, starting in the initial situation S_0 , prove $(\exists s)Do(program, S_0, s)$.

GOLOG: A Novel Logic Programming Language

- GOLOG = alGOl in LOGic.
- Situation calculus semantics (first order logic + some second order).
- The GOLOG interpreter is a theorem-prover.
- Background axioms:

- Initial state of the world:

$$\begin{aligned} &ontable(A, S_0), \text{ blue}(B, S_0) \vee \text{ red}(B, S_0), \\ &(\forall x).ontable(x, S_0) \supset \text{ green}(x, S_0). \end{aligned}$$

- Action precondition axioms, one for each primitive action.
- Successor state axioms, one for each fluent. (These solve the frame problem for primitive actions.)

- Executing *program*:

$$Axioms \vdash (\exists s) Do(program, S_0, s).$$

- Use answer extraction on s to obtain an execution trace, representing the sequence of actions the agent must perform.
- cf. Prolog.

H. Levesque, R. Reiter, Y. Lespérance, F. Lin and R. Scherl. GOLOG: a logic programming language for dynamic domains. *J. Logic Programming*, to appear.

Using GOLOG: An Example Robot Controller

- A mail-delivery robot.

- Primitive actions:

pick_up(pkg), drop_off(pkg), goto(loc), etc.

- Fluents:

recipient(pkg, s), loc(x, s), Requires_Shipment(pkg, s), etc.

- Action Precondition Axioms:

$$Poss(pick_up(pkg), s) \equiv Requires_Shipment(pkg, s) \wedge pos(Robot, s) = Mail_room.$$

- Successor state axioms:

$$\begin{aligned} Poss(a, s) \supset [pos(x, do(a, s)) = p \equiv \\ [x = Robot \vee Robot_Carrying(x, s)] \wedge a = goto(p) \vee \\ pos(x, s) = p \wedge \\ [x \neq Robot \wedge \neg Robot_Carrying(x, s) \vee (\forall p') a \neq goto(p')]]. \end{aligned}$$

- Initial situation:

*pos(Robot, S₀) = (5, 4, 90°), Requires_Shipment(P₁, S₀),
recipient(P₁) = Hector, etc.*

High-Level Control for the Robot

- High-level control specified by GOLOG procedures:

```
proc deliver
  while  $(\exists p) \text{Requires\_Shipment}(p)$  do
     $(\pi r)[(\exists p). \text{Requires\_Shipment}(p) \wedge \text{recipient}(p) = r]?$ ;
    serve( $r$ )
  endWhile
endProc

proc serve( $r$ )
  goto(Mail_room);
  grab_all_packages_for( $r$ );
  goto(loc(in_box( $r$ )));
  drop_off_all_packages_for( $r$ )
endProc

...
```

- Off line execution: Pose

$$\text{Axioms} \vdash (\exists s) \text{Do}(\text{deliver}, S_0, s).$$

- If successful, use answer extraction on s to get:

$$s = \text{do}(A_n, \dots \text{do}(A_2, \text{do}(A_1, S_0)) \dots)$$

- Then emit actions $A_1, A_2, \dots, A_n$ to robot executor.
- This program has successfully controlled an RWI B21 autonomous robot at the University of Toronto, and *the same program* has controlled a Nomad 2000 robot at York University.

Remarks

- An interpreter for GOLOG is a theorem prover for classical, second order logic. The theorem to be proved is obtained by macro-expansion of the program, and, because of while loops and recursion, this may be a second order sentence.
- Fortunately, one can *derive*, from the second order definitions of GOLOG, first order sentences that define a sound (but certainly not complete) interpreter.
- For example, one can derive the following equivalence for non-deterministic iteration:

$$Do(\delta^*, s, s') \equiv s = s' \vee Do(\delta ; \delta^*, s, s').$$

This means that we can design a *sound* GOLOG interpreter for nondeterministic iteration (and hence for **while** loops), and *this interpreter appeals only to first order theorem proving*.

$$\text{do}(\text{star}(E), S, S1) :- S = S1 ; \text{do}(E : \text{star}(E), S, S1).$$

- GOLOG interpreter = *classical* first order theorem prover.
- More precisely,
GOLOG interpreter =
 rewriting to expand the program operators (; | * etc) +
 theorem proving about the domain specific actions and
 relations.

Theorem-Proving about Actions and Fluents

- As everyone knows, general purpose theorem-proving is computationally hopeless; so why has the GOLOG strategy been effective?

- Action precondition axioms are equivalences, and so, act like definitions:

$$Poss(A(\vec{x}), s) \equiv \Psi(\vec{x}, s).$$

- Successor state axioms (which solve the frame problem for primitive actions) have a special syntactic form:

$$Poss(a, s) \supset [F(\vec{x}, do(a, s)) \equiv \phi(\vec{x}, a, s)].$$

So fluents are like *definitions*.

- Theorem proving in GOLOG uses *regression* (= rewriting) wrt these successor state and action precondition axioms to reduce queries about the current situation S , e.g.

$$(\exists p). Robot_is_Carrying(p, S) \wedge recipient(p, S) = r$$

to a theorem proving task *about the initial situation*.

- Theories about the initial situation tend to be very simple, e.g. sets of ground literals (= open world *databases*).

- So we have:

$$GOLOG = \text{database} + \text{definitions} \approx \text{logic programs}.$$

- In fact, Prolog is our implementation language for GOLOG, essentially to do the regression.

A Prolog Interpreter for GOLOG

```
:- op(800, xfy, [&]). /* Conjunction */
:- op(850, xfy, [v]). /* Disjunction */
:- op(870, xfy, [=>]). /* Implication */
:- op(950, xfy, [:]). /* Action sequence */
:- op(960, xfy, [#]). /* Nondeterministic action choice */

do(E1 : E2,S,S1) :- do(E1,S,S2), do(E2,S2,S1).
do(?P),S,S) :- holds(P,S).
do(E1 # E2,S,S1) :- do(E1,S,S1) ; do(E2,S,S1).
do(if(P,E1,E2),S,S1) :- do((?P) : E1) # (?(-P) : E2),S,S1).
do(star(E),S,S1) :- S1 = S ; do(E : star(E),S,S1).
do(while(P,E),S,S1):- do(star(?P) : E) : ?(-P),S,S1).
do(pi(V,E),S,S1) :- sub(V,_,E,E1), do(E1,S,S1).
do(E,S,S1) :- proc(E,E1), do(E1,S,S1).
do(E,S,do(E,S)) :- primitive_action(E), poss(E,S).

/* sub(Name,New,Term1,Term2): Term2 is Term1 with Name replaced by New. */

sub(X1,X2,T1,T2) :- var(T1), T2 = T1.
sub(X1,X2,T1,T2) :- not var(T1), T1 = X1, T2 = X2.
sub(X1,X2,T1,T2) :- not T1 = X1, T1 =..[F|L1], sub_list(X1,X2,L1,L2),
                    T2 =..[F|L2].
sub_list(X1,X2,[],[]).
sub_list(X1,X2,[T1|L1],[T2|L2]) :- sub(X1,X2,T1,T2), sub_list(X1,X2,L1,L2).

holds(P & Q,S) :- holds(P,S), holds(Q,S).
holds(P v Q,S) :- holds(P,S); holds(Q,S).
holds(P => Q,S) :- holds(-P v Q,S).
holds(-P,S) :- not holds(P,S). /* Negation by failure */
holds(some(V,P),S) :- sub(V,_,P,P1), holds(P1,S).

holds(P,S) :- restoreSitArg(P,S,F), F ;
              not restoreSitArg(P,S,F), not (P = -W ; P = (W1 & W2) ;
              P = (W1 => W2) ; P = (W1 v W2) ; P = some(X,W)), P.

restoreSitArg(poss(A),S,poss(A,S)).
```

Limitations of the Prolog Implementation

- Closed World Assumption is built into Prolog via negation-as-failure.
- Fine for some applications where complete information is available about the initial situation.
- Inappropriate for open worlds like robotics. Otherwise, why would the robot ever need to sense the world?
- One approach to open worlds that we are exploring: Use regression (= rewriting) to reduce theorems about the future to theorems about the initial situation. Use a general first-order theorem-prover to prove truths about the initial situation.

Open world GOLOG = Regression + ATP for initial situation.

Recently, this has been used successfully in a planning system for incomplete worlds that is implemented in open-world GOLOG.

Inductive Proofs of GOLOG Program Properties

- Induction principle for **while** loops. Derivable from the GOLOG macro definition for **while**.

$$\begin{aligned} (\forall P, s). P(s) \wedge (\forall s', s'')[P(s') \wedge \phi[s'] \wedge Do(\alpha, s', s'') \supset P(s'')] \\ \supset (\forall s'). Do(\mathbf{while} \ \phi \ \mathbf{do} \ \alpha \ \mathbf{endWhile}, s, s') \supset P(s'). \end{aligned}$$

c.f. Loop invariants in conventional program proofs.

- Example: A blocks world.

Actions

- $move(x, y)$: Move the block x onto block y , provided both x and y are clear.
- $movetotable(x)$: Move the block x onto the table, provided x is clear.

Fluents

- $clear(x, s)$: Block x has no other blocks on top of it, in situation s .
- $on(x, y, s)$: Block x is on (touching) block y , in situation s .
- $ontable(x, s)$: Block x is on the table, in situation s .

- Axiomatization:

Action Precondition Axioms

$$Poss(move(x, y), s) \equiv clear(x, s) \wedge clear(y, s) \wedge x \neq y,$$

$$Poss(movetotable(x), s) \equiv clear(x, s) \wedge \neg ontable(x, s).$$

Successor State Axioms

$$\begin{aligned}
 Poss(a, s) \supset [clear(x, do(a, s)) \equiv & \\
 & a = movetotable(x) \vee (\exists y)a = move(x, y) \vee \\
 & (\exists y)[on(y, x, s) \wedge (a = movetotable(y) \vee \\
 & (\exists z)a = move(y, z))] \vee \\
 & clear(x, s) \wedge \neg(\exists y)a = move(y, x)], \\
 Poss(a, s) \supset [on(x, y, do(a, s)) \equiv & a = move(x, y) \vee \\
 & on(x, y, s) \wedge a \neq movetotable(x) \wedge \neg(\exists z)a = move(x, z)], \\
 Poss(a, s) \supset [ontable(x, do(a, s)) \equiv & a = movetotable(x) \vee \\
 & ontable(x, s) \wedge \neg(\exists y)a = move(x, y)].
 \end{aligned}$$

- A GOLOG program that nondeterministically moves blocks around, so long as there are at least two blocks on the table:

$\gamma = \mathbf{while} \ (\exists x, y)[ontable(x) \wedge ontable(y) \wedge x \neq y] \ \mathbf{do}$
 $\quad (\pi u)movetotable(u) \mid (\pi u, v)move(u, v)$
 $\mathbf{endWhile}$

- Prove that whenever this program terminates, it does so with a unique block on the table, provided there was some block on the table to begin with:

$$\begin{aligned}
 (\forall s, s'). Do(\gamma, s, s') \supset \\
 [(\exists x)ontable(x, s) \supset (\exists! y)ontable(y, s')].
 \end{aligned}$$

- There are also derived induction principles for recursive GOLOG procedures, and the ability to inductively prove correctness properties of these.
- We are especially interested in proofs of reliability for robotics, and more generally, for knowledge-based dynamical systems.